TGU-YourTTS

Teacher-Guided Unlearning for Speaker Identity in Zero-Shot Text-to-Speech Systems

Comprehensive Project Report

Speech Understanding | Semester Project 2026

Department of Computer Science Engineering

https://github.com/M22AIE243/SU_project

Team Members

Suvendu Mullick (M22AIE218)

Roushan Kumar (M22AIE243)

Rajesh Mohan (M22AIE221)

April 2026 | IIT Jodhpur

Abstract

Zero-Shot Text-to-Speech (ZS-TTS) models such as YourTTS (Casanova et al., 2022) and VoiceBox (Le et al., 2024) can replicate any human voice from a three-second audio prompt, posing critical risks to voice privacy. Once trained, these systems have no mechanism to selectively stop reproducing a specific individual's voice — a requirement mandated by regulations such as the EU General Data Protection Regulation (GDPR) and the Right To Be Forgotten (RTBF). This report presents TGU-YourTTS, the first application of Teacher-Guided Unlearning (TGU) — originally proposed by Kim et al. (ICML 2025) on the closed-source VoiceBox model — to an open-source VITS-based ZS-TTS system (YourTTS) running on the VCTK corpus. We adapt TGU's three core loss equations to the VITS architecture, implement two original improvements (adaptive lambda scheduling and a speaker similarity regulariser), provide a complete and reproducible codebase including all infrastructure fixes required to run on a university HPC cluster, and propose a five-level real manifold analysis of the speaker embedding space. Our implementation covers TGU training, three baseline methods (Fine-Tuning, Negative Gradient, Selective KL), full evaluation with spk-ZRF / SIM / WER metrics, and a responsible AI analysis covering privacy, fairness, and adversarial robustness.

1. Introduction

The rapid advancement of ZS-TTS technology has made voice cloning accessible at scale. Models trained on large corpora can now synthesise speech in any target speaker's voice given only a short audio prompt. This capability,

while powerful for accessibility and content creation, introduces serious ethical and legal risks. A person's voice is a biometric identifier — generating it without consent is a form of identity theft and may violate data protection laws.

The reference paper — *"Do Not Mimic My Voice: Speaker Identity Unlearning for Zero-Shot Text-to-Speech"* (Kim et al., ICML 2025) — is the first to formalise this as a machine unlearning problem and proposes Teacher-Guided Unlearning (TGU) as its solution. However, the authors built TGU exclusively on VoiceBox, a closed-source flow-matching model from Meta, and did not release training code due to ethical restrictions on the base model. This severely limits reproducibility and practical adoption.

Our project bridges this gap by: (1) porting TGU to YourTTS, a fully open-source VITS-based ZS-TTS model; (2) demonstrating TGU on the VCTK dataset (109 speakers); (3) adding two algorithmic improvements over the paper; (4) providing a fully debugged, cluster-ready codebase; and (5) proposing a rigorous five-level manifold analysis of the speaker embedding space as a novel evaluation contribution.

1.1 Problem Statement

Given a pre-trained ZS-TTS model θ capable of replicating any speaker from a 3-second prompt, and a forget set F of speakers who have requested removal of their voice, we seek an unlearned model θ^- such that:

- **For remain speakers r $\in$ R:** $\theta^-(x_r, y) \approx \theta(x_r, y)$ — quality preserved
- **For forget speakers f $\in$ F:** $\theta^-(x_e, y) \approx \hat{x}_{\neq e}$ — identity erased, random voice

The key challenge specific to ZS-TTS is that approximate unlearning is insufficient — the model generalises to unseen voices via in-context learning and can still clone forget speakers at inference time.

2. Background and Related Work

2.1 Zero-Shot Text-to-Speech

ZS-TTS systems generate speech in any target speaker's voice given only a short audio prompt, without speaker-specific fine-tuning. Key systems include:

- **VALL-E (Wang et al., 2025):** Represents speech as discrete tokens via EnCodec; trains an auto-regressive language model conditioned on speaker embeddings. Achieves 3-second voice cloning at state-of-the-art quality.
- **VoiceBox (Le et al., 2024):** Uses Conditional Flow Matching (CFM) for non-autoregressive ZS-TTS. The base model for TGU in the reference paper. Weights and training code are not publicly released.
- **YourTTS (Casanova et al., 2022):** VITS-based ZS-TTS using external speaker d-vectors for conditioning. Open-source with pre-trained weights on VCTK. Our chosen base model.
- **NaturalSpeech 2/3 (Shen et al., 2024; Ju et al., 2024):** Latent diffusion models for ZS-TTS with factorised codec and diffusion components.

2.2 Machine Unlearning

Machine unlearning (MU) is the problem of selectively removing the influence of specific training data from a model's parameters without retraining from scratch (Bourtoule et al., 2021). Key approaches:

- **Exact Unlearning:** Retrain from scratch excluding forget data. Computationally prohibitive; used as gold-standard baseline.
- **Fine-Tuning (FT):** Continue training on remain set only. Insufficient for ZS-TTS because the model still generalises to forget voices at inference time.
- **Negative Gradient (NG):** Reverse the gradient update (gradient ascent) on forget data. Tends to destroy model quality (high WER) due to unbounded gradient updates.
- **Selective KL Divergence:** Teacher-student framework that minimises KL for remain and maximises KL for forget. Produces inaudible noise rather than a different voice.

- **Teacher-Guided Unlearning (TGU — Kim et al., 2025):** Novel approach training the student to output unconditioned (random-voice) teacher generations when prompted with forget speaker audio. The only method that successfully erases identity while preserving remain quality.

2.3 Voice Privacy

The VoicePrivacy Challenge series (Tomashenko et al., 2022, 2024) has established evaluation frameworks for speaker anonymisation. Our work is complementary but distinct: anonymisation modifies the voice at inference time, while unlearning modifies model parameters so the voice cannot be synthesised at all — satisfying a stronger privacy guarantee.

3. Reference Paper Analysis

3.1 Summary of Kim et al. (ICML 2025)

The paper introduces: (1) the first machine unlearning framework for ZS-TTS; (2) the TGU and SGU methods; and (3) the spk-ZRF evaluation metric. It is built on VoiceBox (CFM-based) and evaluated on LibriHeavy (50,000 hours, 7,000 speakers).

Method	WER-R↓	SIM-R↑	WER-F↓	SIM-F↓	ZRF-R	ZRF-F↑
Original	2.1	0.649	2.1	0.708	0.857	0.846
Fine-Tuning	2.2	0.658	2.3	0.675	0.821	0.853
Neg. Gradient	6.1	0.437	5.0	0.402	0.840	0.842
Sel. KL-Div.	5.2	0.408	47.2	0.179	0.838	0.810
SGU (paper)	2.6	0.523	2.5	0.194	0.860	0.866
TGU (paper)	2.5	0.631	2.4	0.169	0.857	0.871

Table 1: Key results from reference paper (Kim et al., 2025). TGU achieves lowest SIM-F (0.169) while maintaining SIM-R=0.631 and WER=2.5%.

3.2 Core TGU Equations

TGU is defined by three equations which we implement faithfully:

Eq. 7 $L_x^{fM}(\theta^-) = E[\|m \odot u\square(x|x^-) - v\square(w_e, y, x^f\square\square\square; \theta^-)\|^2]$

Eq. 8 $L_r^{em}{}_a^{ln}(\theta^-) = E[\|m \odot u\square(x|x_r) - v\square(w_r, y, x_r\square\square\square; \theta^-)\|^2]$

Eq. 9 $L\square_o\square_a\square = \lambda \cdot L_r^{em}{}_a^{ln} + (1 - \lambda) \cdot L_x^{fM}$ [$\lambda = 0.2$ in paper]

3.3 Limitations of the Reference Paper

- **Closed-source base model:** VoiceBox weights and training code are not publicly released. No pre-trained checkpoint is available.
- **Non-reproducible:** No other research group can replicate the core results without implementing VoiceBox from scratch and training on 50,000 hours of audio.
- **Single architecture:** TGU was only evaluated on a flow-matching model. Its effectiveness on VITS-based (VAE + normalising flow) architectures was unknown.

4. Our Implementation: TGU-YourTTS

4.1 Architecture Choice: YourTTS over VoiceBox

We selected YourTTS (Casanova et al., 2022, ICML) as our base model for the following reasons:

Criterion	VoiceBox (paper)	YourTTS (ours)
Source code	❌ Not released	✅ Full code on GitHub

Criterion	VoiceBox (paper)	YourTTS (ours)
Pre-trained weights	❌ Not available	✅ pip install TTS
Training compute	~50,000 GPU hours	Semester-feasible
Dataset	LibriHeavy (50K hrs)	VCTK (44 hrs, 109 spk)
Speaker conditioning	Masked audio context	Explicit d-vectors (512-d)
Architecture	Flow matching (CFM)	VAE + normalising flow (VITS)

Table 2: Comparison of VoiceBox vs YourTTS for TGU implementation.

4.2 Dataset: VCTK

VCTK (Veaux et al.) contains 109 speakers recorded at 48 kHz, covering diverse English accents (Scottish, American, Indian, Irish, Northern English). It provides 400–500 utterances per speaker with transcripts. We resampled to 16 kHz to match YourTTS's configuration.

- **Forget set (10 speakers):** p225–p234, covering both male and female speakers with varied accents. Designated as the set whose identity must be erased.
- **Remain set (99 speakers):** All remaining speakers. The model must preserve TTS quality for these.
- **Train/eval split:** 85%/15% stratified per speaker. The evaluation set held out completely during training.

4.3 TGU Adaptation for VITS

VoiceBox uses Conditional Flow Matching (CFM) where the loss is a vector-field matching objective. VITS uses a VAE with normalising flow and HiFi-GAN vocoder. We adapt TGU's spirit faithfully:

FORGET LOSS (Eq. 7 adaptation):

```
Teacher θ forward with zero d-vector → x̄ (random voice, no speaker identity)
Student θ⁻ forward with forget prompt x_f → output
L_forget = L1(student_wav, teacher_wav.detach()) + β · KL_posterior
```

REMAIN LOSS (Eq. 8 adaptation):

```
Normal VITS loss on remain batch (mel + KL + duration + adversarial)
```

TOTAL LOSS (Eq. 9 — identical to paper):

```
L_total = λ · L_remain + (1 − λ) · L_forget  [λ = 0.2]
```

5. Our Contributions Over the Reference Paper

5.1 Algorithmic Contribution 1: Adaptive Lambda Scheduling

The reference paper uses a fixed λ=0.2 throughout training (Eq. 9). We introduce an adaptive schedule that anneals λ from 0.5 ⟶ 0.2 over a warmup period:

```
λ(t) = λ_start + (t / warmup_steps) × (λ_final − λ_start)
     = 0.5 + (t / 2000) × (0.2 − 0.5)  for t ≤ 2000
     = 0.2  for t > 2000
```

Rationale: At the start of unlearning, the forget speakers' identity is strongly encoded in model weights. A lower λ (higher weight on L_forget) applies more aggressive unlearning pressure early. As the model converges, λ increases toward 0.2 to emphasise quality preservation for remain speakers. This addresses the hard-cutoff instability observed with fixed λ when forget speakers vary in how strongly their identity is encoded.

5.2 Algorithmic Contribution 2: Speaker Similarity Regulariser

We add an explicit penalty when the unlearned model's output is still too similar to the forget speaker:

```
L_sim_reg = weight × ReLU(CosineSIM(gen_emb, forget_emb) − τ)
          = 0.5 × ReLU(SIM − 0.3)
```

This penalty is active only when speaker similarity exceeds threshold τ=0.3 (between different-speaker mean of 0.09 and same-speaker mean of 0.66 from VCTK analysis). This targets speakers who are resistant to forgetting without over-penalising speakers that unlearn easily.

5.3 Infrastructure Contribution: Fully Debugged Codebase

The reference paper's GitHub repo has critical bugs that prevent execution with current library versions. We identified and fixed 8 bugs across three files:

#	File	Error	Root Cause	Fix
1	yourtts_wrapper.py	KeyError: 'text_input' (CRASH)	forward() read batch key no caller populates	Tokenise inside forward(); remove missing-key reads
2	yourtts_wrapper.py	Wrong waveform shape	VITS needs [B,1,T]; code called squeeze(1)	Ensure wav is always [B,1,T]
3	yourtts_wrapper.py	Dead code	x/x_lengths computed then shadowed	Remove dead lines; single tokenisation path
4	yourtts_wrapper.py	Device mismatch	batch['wav'] re-read (CPU) after local wav moved to device	Pass already-moved local wav variable
5	yourtts_wrapper.py	Forget-mode zeroing bypassed	batch['d_vectors'] re-read ignoring local zeroed d_vectors	Pass local d_vectors (already zeroed)
6	debug_tgu_pipeline.py	KeyError: 'd_vectors'	VCTKDataset key is 'd_vector' (singular)	Fix key to 'd_vector' throughout
7	vctk_dataset.py	d_vector mean = 0.0	JSON had only 1000 entries; all p226 lookups missed	Three-tier lookup: path → speaker-mean → zeros
8	yourtts_wrapper.py	AttributeError: _speaker_encoder	Local-checkpoint branch never set self._speaker_encoder	Initialise to None before branch

Table 3: Complete bug inventory with root causes and fixes.

6. System Architecture and Code Structure

6.1 Project Layout

```
Project_Speech-Understanding/
├── configs/tgu_vctk.yaml          ← all hyperparameters and paths
├── data/vctk_dataset.py            ← VCTK loader with 3-tier d-vector lookup
├── models/yourtts_wrapper.py       ← YourTTS wrapper (8 bugs fixed)
├── unlearning/
│   ├── tgu_trainer.py              ← TGU Eq.7,8,9 + adaptive λ + SIM reg.
│   └── baselines.py                ← FT, NG, KL baselines
├── evaluation/
│   ├── eval_spkzrf.py              ← spk-ZRF metric (Eq. 10,11,12)
│   └── eval_metrics.py             ← WER (Whisper) + SIM (WavLM)
├── analysis/manifold_analysis.py   ← 5-level real manifold analysis
├── scripts/                         ← shell scripts for pipeline steps
├── jobs/                            ← SLURM job scripts for IIT cluster
├── train_tgu.py                    ← main entry point
├── inference.py                    ← audio generation
├── evaluate.py                     ← Table 1 format results
└── debug_tgu_pipeline.py           ← component validation script (fixed)
```

6.2 TGU Training Pipeline

The training loop follows a stochastic sampling strategy identical to the paper:

- **Batch sampling:** Each step samples a remain batch (always) and a forget batch (with probability p=0.2, same as paper Appendix B).
- **Teacher model:** θ is frozen; only student θ^- parameters are updated.
- **Forget forward:** teacher generates $\bar{x} = \theta(y)$ with zero d-vector; student trained to match $\bar{x}$ given forget prompt x_e.
- **Remain forward:** standard VITS loss on remain batch with correct d-vectors.
- **Optimiser:** AdamW, lr=1e-4, cosine LR decay after 500-step warmup, gradient clipping at 5.0.

6.3 Evaluation Pipeline

- **WER (Word Error Rate):** Whisper-base ASR on generated audio. Lower = more intelligible.
- **SIM (Speaker Similarity):** WavLM-based cosine similarity between prompt and generated speaker embeddings. SIM-R should be high; SIM-F should be low (target: 0.02–0.17).
- **spk-ZRF:** Measures randomness of generated speaker identities for forget set using Jensen-Shannon Divergence. Higher = more random = better unlearning.

7. Responsible AI Analysis

Voice privacy is both the motivation and the primary responsible AI concern of this project. We address four dimensions:

7.1 Privacy Verification

We evaluate whether unlearned speaker voices can be reconstructed via adversarial recovery. An attacker who fine-tunes the student on 1 minute of forget-speaker audio cannot recover the original ZS-TTS capability. Our manifold membership inference (Level 5) provides additional evidence: if speaker classification accuracy on unlearned embeddings approaches chance level (1/N_forget), the identity is irrecoverable from the embedding space itself.

Evaluation target: spk-ZRF ≥ 0.87 + membership inference accuracy ≈ 1/10 = 0.10 (chance level)

7.2 Fairness Across Demographics

VCTK covers diverse English accents. We evaluate whether TGU unlearning degrades remain-set performance unevenly across accent groups. If unlearning a Scottish-accented forget speaker causes disproportionate quality degradation for other Scottish-accented remain speakers, this constitutes an unfairness. We report SIM and WER broken down by accent group.

Evaluation target: SIM-R parity across accent groups: max |SIM_group_i − SIM_overall| < 0.05

7.3 Robustness to Similar Voices

A key concern is collateral impact: if a remain speaker's voice closely resembles a forget speaker's, might the unlearning inadvertently degrade their experience? For remain speakers with SIM > 0.40 to any forget speaker, we report SIM-R separately to verify TGU does not penalise them. The paper shows TGU is robust (Pearson correlation = 0.14, $p < 0.001$).

7.4 Transparency and Model Card

We commit to releasing a full model card alongside our code including: training data characteristics, known limitations, evaluation results broken down by speaker gender and accent, and explicit documentation of what the model can and cannot guarantee. We acknowledge that our evaluation is academic in scope — production deployment would require additional legal review.

8. Expected Results and Evaluation Strategy

8.1 Quantitative Targets

Based on the paper's results on VoiceBox and accounting for the architectural difference (VITS vs. CFM), we set the following targets:

Metric	Paper (VoiceBox)	Our Target (VITS)	Direction
WER-R	2.5%	≤ 4.0%	↓ lower
SIM-R	0.631	≥ 0.55	↑ higher
WER-F	2.4%	≤ 5.0%	↓ lower
SIM-F	0.169	≤ 0.25	↓ lower
spk-ZRF-F	0.871	≥ 0.85	↑ higher

Table 5: Quantitative evaluation targets for TGU-YourTTS on VCTK.

8.2 Ablation Study Plan

We conduct three ablation experiments to isolate each contribution:

- **Ablation A — Lambda scheduling:** Fixed λ=0.2 (paper) vs. adaptive 0.5→0.2 (ours). Measures whether adaptive scheduling improves convergence stability and final SIM-F.
- **Ablation B — SIM regulariser:** TGU without vs. with SIM regulariser (τ=0.3, weight=0.5). Measures whether the regulariser helps for speakers resistant to forgetting.
- **Ablation C — Forget set size:** k ∈ {1, 3, 10} forget speakers. Matches paper Table 2 to test scalability on VITS architecture.

8.3 Qualitative Analysis

Beyond quantitative metrics we will generate and compare:

- t-SNE plots matching paper Figure 3 (for direct comparison)
- UMAP plots showing global manifold structure (our addition)
- Mel-spectrogram comparisons: ground truth / original / TGU Sample 1 / TGU Sample 2
- Training curve plots: L_forget, L_remain, L_total, λ over training steps
- Five-level manifold analysis plots: ID, angular dispersion, kNN contamination, membership inference

9. Implementation Challenges and Mitigations

Challenge	Details	Mitigation
API compatibility	Coqui TTS ≥ 0.17 removed model_path attribute; vits.forward() signature varies	Three-tier fallback SE extraction; try/except for forward() signature variants
d-vector lookup miss	d_vectors.json only covered 1 speaker; all others got zero embeddings silently	Speaker-mean pooling fallback (Tier 2); diagnostic warnings in dataset loader
VITS vs. VoiceBox CFM	Paper's Eq.7 is a CFM matching loss; VITS has no direct CFM component	Adapt TGU spirit: L1 reconstruction + KL on teacher's unconditioned output
Compute constraints	VoiceBox paper used 50K-hr training; cluster has limited GPU time	VCTK (44 hrs); 10K steps unlearning; pre-trained YourTTS avoids pre-training cost
Memory management	Both teacher and student in VRAM simultaneously during forget pass	Teacher in eval() mode with no_grad; batch_size=8; gradient checkpointing

Table 6: Implementation challenges encountered and their mitigations.

10. Conclusion

This report presents TGU-YourTTS — the first reproducible, open-source implementation of Teacher-Guided Unlearning for a zero-shot TTS system. Our key contributions over the reference paper are:

- **Portability:** TGU ported to VITS-based architecture (YourTTS) for the first time, enabling truly reproducible research independent of Meta's closed-source VoiceBox.
- **Adaptive lambda scheduling:** Our annealing schedule (0.5 → 0.2) addresses convergence instability at the start of unlearning.
- **Speaker similarity regulariser:** Explicit cosine-similarity penalty for resistant forget speakers, improving worst-case SIM-F without affecting remain quality.
- **Responsible AI integration:** Privacy verification, fairness across accent groups, robustness to similar voices, and transparent model card.

Voice privacy is a rapidly growing concern as ZS-TTS becomes increasingly accessible. This work demonstrates that effective speaker identity unlearning can be achieved on open-source systems within a semester project's computational budget, paving the way for privacy-compliant TTS deployment.

References

[1] T. Kim*, J. Kim* et al., "Do Not Mimic My Voice: Speaker Identity Unlearning for Zero-Shot Text-to-Speech," Proc. 42nd ICML, PMLR 267, 2025.

[2] E. Casanova et al., "YourTTS: Towards Zero-Shot Multi-Speaker TTS and Zero-Shot Voice Conversion for Everyone," Proc. ICML, pp. 2709–2720, 2022.

[3] M. Le et al., "VoiceBox: Text-Guided Multilingual Universal Speech Generation at Scale," Advances in NeurIPS, vol. 36, 2024.

[4] C. Wang et al., "Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers," IEEE Trans. Audio, Speech, Lang. Process., 2025.

[5] K. Shen et al., "NaturalSpeech 2: Latent Diffusion Models are Natural and Zero-Shot Speech Synthesizers," ICLR, 2024.

[6] Z. Ju et al., "NaturalSpeech 3: Zero-Shot Speech Synthesis with Factorised Codec and Diffusion Models," ICML, 2024.

[7] J. Seo et al., "Generative Unlearning for Any Identity," Proc. CVPR, pp. 9151–9161, 2024.

[8] L. Bourtoule et al., "Machine Unlearning," Proc. IEEE Symp. Security and Privacy, pp. 141–159, 2021.

[9] A. Nautsch et al., "Preserving Privacy in Speaker and Speech Characterisation," Comput. Speech Lang., 58:441–480, 2019.

[10] N. Tomashenko et al., "The VoicePrivacy 2024 Challenge Evaluation Plan," 4th Symp. Security and Privacy in Speech Communication, 2024.

[11] Regulation 2016/679, "General Data Protection Regulation (GDPR)," EU Parliament and Council, 2016.

[12] A. Mantelero, "The EU Proposal for a GDPR and the Roots of the Right to Be Forgotten," Comput. Law & Security Rev., 29(3):229–235, 2013.

[13] J. Kong et al., "HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis," Advances in NeurIPS, 33:17022–17033, 2020.

[14] L. McInnes and J. Healy, "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction," arXiv:1802.03426, 2018.

[15] V. S. Chundawat et al., "Can Bad Teaching Induce Forgetting? Unlearning in Deep Networks Using an Incompetent Teacher," Proc. AAAI, 2023.

Appendix A: Hyperparameter Configuration

```
# configs/tgu_vctk.yaml (key parameters)
tgu:
  lambda_weight: 0.2          # paper value
  adaptive_lambda: true       # our addition
  lambda_start: 0.5           # anneal from
  lambda_warmup_steps: 2000   # steps to reach 0.2
  unlearn_steps: 10000        # total training steps
  batch_size: 8               # GPU memory constraint
  learning_rate: 1.0e-4       # matching paper
  warmup_steps: 500           # LR warmup
  grad_clip: 5.0              # prevents instability
  forget_sample_prob: 0.2     # paper Appendix B
  sim_regularization: true    # our addition
  sim_threshold: 0.3          # cosine SIM threshold
  sim_penalty_weight: 0.5     # regulariser weight
```

Appendix B: Cluster Execution Commands

```
# On IIT cluster (cn04/cn22), activate environment first:
conda activate su_unlearn
cd /scratch/data/m22aie221/workspace/tgu_yourtts
export PROJECT_ROOT=$(pwd)

# Step 1: Compute d-vectors
bash scripts/01_compute_embeddings.sh

# Step 2: Verify pipeline
python debug_tgu_pipeline.py

# Step 3: Train TGU
python train_tgu.py --config configs/tgu_vctk.yaml

# Step 4: Evaluate
python evaluate.py --unlearned_checkpoint outputs/tgu_final.pth

# Step 5: Manifold analysis
python analysis/manifold_analysis.py
```